

La versión local por dentro: las *herramientas* antes de empezar

El instalable de Windows parece simple por fuera: eliges variable, departamento y años, y aparecen tus archivos. Por dentro trabaja un **equipo de librerías de Python**, cada una con un oficio. Esta primera hoja presenta al equipo; la segunda muestra el viaje completo del dato.



¿Qué es una librería? Es una caja de herramientas de código que otras personas ya construyeron y comparten. Como al armar un mueble: no fabricas el destornillador, lo usas. El proyecto combina estas 10 herramientas y les pone encima su propia receta, la lógica que las coordina.

EL ELENCO

Diez herramientas, un oficio cada una



python-dotenv **el llavero**

Lee la llave de acceso (App Token) guardada en un archivo aparte llamado .env. Así el secreto nunca queda escrito dentro del programa.



sodapy **el traductor con teléfono directo**

Ya sabe hablar con Socrata, la plataforma donde el IDEAM publica sus datos. Le pasas el filtro y ella arma la petición correcta.



requests **el mensajero**

Envía y recibe información por internet, igual que hace tu navegador al abrir una página, pero en silencio y bajo control del programa.



concurrent.futures **el equipo de mensajeros**

Permite bajar varios bloques de datos al mismo tiempo, como abrir varias cajas en el supermercado en vez de hacer una sola fila.



pandas **la hoja de cálculo inteligente**

Maneja los datos como una tabla con superpoderes: convierte fechas y números a su forma correcta, filtra y transforma columnas enteras de una vez.



hashlib **el sello de identidad**

Calcula una huella digital única para cada medición (estación + sensor + fecha). Con esa huella se detectan y eliminan los datos repetidos.



pyarrow **la empacadora al vacío**

Guarda las tablas en formato Parquet, que comprime mucho y se lee rápido. Ideal para Power BI o Python; ocupa poco y conserva todo.



pydantic **el inspector de calidad**

Revisa que los datos cumplan las reglas definidas (tipos, formatos, campos obligatorios) antes de dejarlos pasar en el modo servidor.



rich **el decorador**

Le da color a la terminal: tablas alineadas, paneles con borde y barras de progreso. Sin ella, todo sería texto plano gris.



textual **el arquitecto de la sala bonita**

Construye la interfaz visual de pantalla completa (la TUI): pantallas, botones y listas que se navegan con el teclado, viviendo dentro de la terminal.

1 · Tener Python 3.10 o superior

Es el lenguaje en que está escrito todo. Si usas el **ejecutable .exe** no necesitas ni esto: ya viene con todo adentro, doble clic y listo.

2 · Instalar con un comando

pip install ideam-data-automator descarga el programa y sus 10 herramientas automáticamente, en las versiones correctas.

3 · Un App Token (opcional)

Llave **gratuita** de datos.gov.co que se guarda en el archivo .env. Sin ella funciona igual, pero el portal atiende menos peticiones por hora.

El viaje del dato, paso a paso: de la nube del IDEAM a tu carpeta

Esto es lo que pasa por dentro, en orden, cada vez que pides una descarga. Cada paso indica el archivo del código que hace el trabajo y las herramientas que usa.

01
ENCENDER

Se prepara el tablero de control `config.py`

El programa carga la lista de las **21 variables** disponibles (lluvia, ríos, temperatura...), el mapa de los **33 departamentos** con todas sus formas de escritura (con y sin tilde), y toma del llavero (**python-dotenv**) el App Token que le da más permisos para pedir datos.

02
ELEGIR

Tú dices qué quieres `tui.py · main.py`

Una pantalla amable (hecha con **textual** y **rich**) te pregunta variable, departamentos, años y filtros avanzados. Antes de pedirte los años, consulta cuántas estaciones hay para tu selección y desde qué año tienen datos, para que no pidas algo que no existe.

03
TRADUCIR

Tu pedido se vuelve una pregunta segura `query_validation.py`

El pedido humano se traduce al idioma del portal (**SoQL**): "dame lluvia DONDE departamento sea Atlántico Y fecha entre 2015 y 2020". Se incluyen todas las variantes del nombre (San Andrés aparece bajo casi diez formas distintas), se **blinda la consulta** para que nadie cuele instrucciones maliciosas, y el pedido grande se parte en bloques por año y por grupos de estaciones.

04
DESCARGAR

Los datos viajan por pedacitos y en paralelo `extract.py · core.py · batch.py`

Con **sodapy** y **requests** el programa pide página por página (hasta 50.000 filas cada una) y baja **varios bloques a la vez** gracias al equipo de mensajeros. Si el portal se satura, **espera y reintenta solo** (1, 2, 4, 8 segundos) sin caerse. El modo rápido trae los datos comprimidos y resulta 5 a 10 veces más veloz.

05
LIMPIAR

La estación de calidad `transform.py`

Los datos llegan crudos: fechas como texto, números mezclados, posibles repetidos. **pandas** convierte cada cosa a su tipo correcto, **hashlib** le pone a cada medición su huella única, y se **eliminan los duplicados** que el portal a veces entrega. Si tu archivo tiene menos filas que el portal, no perdiste datos: quitaste repetidos.

06
EMPACAR

Todo ordenado y a prueba de cortes `exporting.py`

La tabla se agrupa por **departamento y municipio** y se guarda en Parquet (con **pyarrow**) y CSV para Excel. La escritura es **atómica**: primero a un archivo temporal y solo al terminar bien se renombra, así un corte de luz nunca deja un archivo dañado. Junto a los datos queda un **RESUMEN.txt** que dice qué trajiste y qué porcentaje de meses tiene dato.

07
LISTO

El resultado en tu carpeta de Documentos

En **Documentos / IDEAM_Data** aparece todo organizado en carpetas, y el programa reporta cuántas filas trajo, cuántos archivos generó y en cuánto tiempo. Lo que antes tomaba horas de trabajo manual queda listo en minutos.